

Wiseswap: Elastic Datacenter Network-Aware Disaggregated Memory for Multi-Tenant Cloud

Mingxuan Liu

Northwestern Polytechnical University
School of Computer Science
Xi'an, Shaanxi, China
liumingxuan@mail.nwpu.edu.cn

Tianhai Zhao

Northwestern Polytechnical University
School of Computer Science
Xi'an, Shaanxi, China
zhaoth@nwpu.edu.cn

Jianhua Gu*

Northwestern Polytechnical University
School of Computer Science
Xi'an, Shaanxi, China
gujh@nwpu.edu.cn

Dong Sun

Northwestern Polytechnical University
School of Computer Science
Xi'an, Shaanxi, China
sundong1118@gmail.com

Abstract

Disaggregated Memory Systems (DMS) hold substantial potential for cloud datacenters but face critical deployment barriers in multi-tenant RDMA environments. Existing DMS designs rely on idealized assumptions—overlooking interference from co-located RDMA applications, oversimplifying fabric topology considerations, and lacking elastic service-level objectives (SLOs) guarantees—resulting in performance degradation and resource inefficiency.

To bridge these gaps, we introduce *Wiseswap*, an elastic, datacenter network-aware DMS that delivers robust memory disaggregation for multi-tenant clouds through three key innovations: (1) *Preemption-enabled isolation*: A low-overhead kernel mechanism utilizes WAIT/ENABLE RDMA primitives to prioritize latency-critical swap operations over user-space RDMA flows, maintaining tenant fairness; (2) *Adaptive fabric path selection*: In-kernel telemetry dynamically probes latency and routes memory traffic through uncongested paths, mitigating interference from elephant flows; (3) *Feedback-directed autoscaling*: Fine-grained optimization of DMS-specific parameters—dynamically optimizes resource allocation under fluctuating workloads, guaranteeing stringent SLOs while minimizing resource overhead. Evaluations demonstrate that *Wiseswap* improves throughput by 1.3–2.4× and reduces tail latency by 57–73% under contention compared to state-of-the-art solutions, while consistently meeting strict SLO targets.

CCS Concepts

• **Computer systems organization** → **Cloud computing**; • **Networks** → **Data center networks**; **Network management**.

Keywords

Disaggregated Memory; RDMA; Multi-Tenant Cloud; Datacenter Network

*Jianhua Gu is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, Dubai, United Arab Emirates.

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792309>

ACM Reference Format:

Mingxuan Liu, Jianhua Gu, Tianhai Zhao, and Dong Sun. 2026. Wiseswap: Elastic Datacenter Network-Aware Disaggregated Memory for Multi-Tenant Cloud. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3774904.3792309>

1 Introduction

Disaggregated Memory Systems (DMS) [2, 3, 6, 11, 13, 22, 25, 39, 40, 42, 46–49, 53, 54] represent a promising architectural shift for modern datacenters, decoupling memory resources from compute nodes to enable flexible resource utilization. By enabling remote memory access with near-local performance via high-speed networks like RDMA, DMS addresses critical limitations of server-centric designs, particularly in memory-bound workloads where scaling often causes resource fragmentation.

Multi-tenant cloud datacenters [5, 7, 10, 37, 38] stand to gain exceptional value from DMS adoption. For cloud providers managing large-scale infrastructures, transparent integration of DMS via kernel modules¹ unlocks substantial capital and operational savings through improved memory utilization and reduced over-provisioning. Concurrently, cloud users benefit transparently: containerized applications [31] or serverless functions [17] experience significant throughput improvements without code modifications, as DMS dynamically scales memory resources beyond physical node constraints. This dual advantage positions DMS as a foundational technology for next-generation cloud infrastructure.

Despite this potential, we have found that state-of-the-art DMSs reveal several limitations when deployed in multi-tenant cloud datacenters. Existing designs rely on idealized assumptions that collapse under multi-tenant pressures, failing to meet user-defined Service Level Objectives (SLOs) that encompass throughput, latency, and dynamic autoscaling requirements. Three critical gaps emerge:

(1) **RDMA NIC contention**: Prior work [3, 13, 42, 47, 49, 53, 54] assumes that DMS applications monopolize RDMA clusters, ignoring interference from tenant RDMA apps sharing RDMA NICs (RNIC) (Fig. 1). Solutions like Canvas [47] address inter-DMS app contention, leaving cross-tenant isolation unaddressed.

¹We focus on kernel-level DMS [3, 13, 39, 42, 47, 49, 53, 54] in this paper, providing transparency different from runtime-level DMS [6, 25, 40].

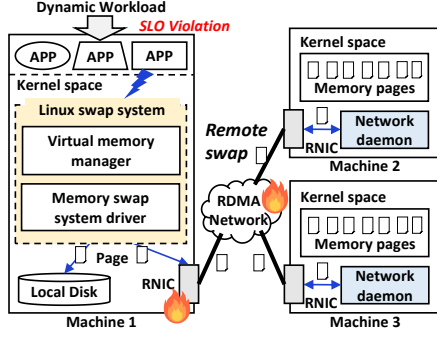


Figure 1: Principle of disaggregated memory system (DMS). The flames highlight performance bottlenecks caused by RNIC contention and RDMA network interference.

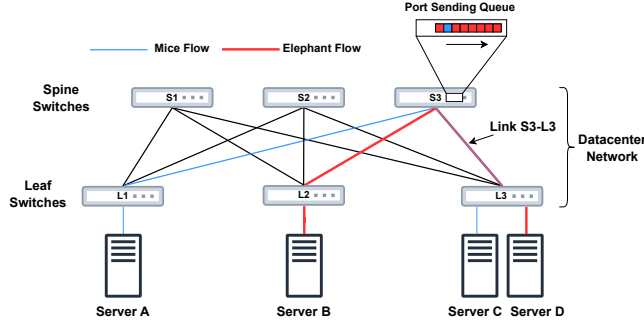


Figure 2: The DMS in a real cloud datacenter is interconnected by Switch Fabric.

(2) **Network Fabric interference:** Prior work [2, 3, 6, 11, 13, 22, 25, 39, 40, 42, 46–49, 53, 54] using simplified datacenter network topologies (Fig. 2) disregards switch-level interference. In such scenarios, throughput-sensitive or bandwidth-sensitive flow (elephant flows) can disrupt latency-sensitive DMS flow (mice flows) through ECMP hash collisions (§2.2.2).

(3) **SLO violations:** Prior work [47] reveals the limitations of static resource configurations, which cannot adapt to cloud workloads’ dynamic, bursty nature. This leads to either SLO breaches or severe resource waste when using conventional autoscalers.

In this paper, we introduce Wisewap to bridge these gaps, a disaggregated memory system co-designed with datacenter networks and multi-tenant cloud requirements. Wisewap integrates three novel mechanisms enabling direct deployment in cloud datacenters: (1) a preemption-enabled RNIC scheduler for tenant fairness, (2) a network fabric-agnostic path selector for load balancing, and (3) an SLO-driven autoscaling engine tailored to DMS characteristics. By holistically managing contention at the RNIC, network fabric, and cluster resource scaling layers, Wisewap maintains performance isolation while ensuring predictable SLOs.

Building Wisewap posed three technical challenges (§4.1): (1) Implementing low-overhead kernel support for preempting RDMA operations to prioritize swap flow, (2) Enabling fabric fairness despite unknown topology and ECMP variations, and (3) Delivering

fine-grained autoscaling for DMS-specific parameters without resource bloat. Our solutions comprise: (1) A WAIT/ENABLE-based preemption scheduling in the kernel I/O path (§5.1); (2) In-kernel telemetry for adaptive path selection via queue pair (QP) pool management (§5.3); (3) Feedback-directed autoscaling with closed-loop control adjusting remote memory, QP counts, and prefetch buffers against SLO targets (§5.2).

Contributions. We highlight the contributions as follows:

Problem: existing DMS designs rely on idealized assumptions that collapse under multi-tenant pressures, failing to meet user-defined SLOs in cloud.

Wisewap: a disaggregated memory system co-designed with datacenter networks and multi-tenant cloud requirements enabling direct deployment in cloud datacenters.

2 Related work

2.1 Kernel-level Disaggregated Memory

Disaggregated Memory Systems (DMS) utilize high-speed RDMA networks to decouple memory from compute nodes², enabling transparent memory expansion for cloud applications. Kernel-level page swapping underpins modern DMS [3, 13, 28–30, 32, 39, 42, 45, 47, 49–51, 53, 54] by intercepting page faults to redirect memory accesses to remote nodes. Fastswap [3] dedicates cores to asynchronous page reclamation, alleviating bottlenecks in Linux’s direct reclamation. Hermit [39] employs feedback-directed asynchrony to offload non-critical operations (e.g., swap-out, accounting), reducing tail latency in Memcached. However, these designs assume single-application contexts and overlook isolation challenges in multi-tenant clouds (e.g., contention and interference).

Isolation becomes critical when multiple applications share RNIC resources. Canvas [47] addresses intra-DMS interference via per-application swap partitions, dedicated prefetchers, and RDMA bandwidth isolation. Yet, it does not account for cross-tenant RDMA applications (e.g., databases) competing for RNIC bandwidth, causing unpredictable latency spikes. *This limitation is acute in cloud environments where RNICs are shared among diverse workloads, violating tenant fairness and SLOs.*

Kernel-level isolation inevitably extends the critical path of remote paging, impairing performance [39, 47]. To compensate, two primary methods are adopted: *prefetching* and *asynchrony*.

(1) **Prefetching optimizations**, such as Leap’s majority-based algorithm [2] and Canvas’s semantics-aware approach [47], anticipate page faults to reduce remote access latency. However, Leap’s shared prefetcher interferes with multi-threaded workloads (e.g., Spark), and Canvas’s runtime-dependent prefetching sacrifices transparency and generality. *Both struggle to adapt to bursty, heterogeneous access patterns in cloud workloads.*

(2) **Asynchronous execution** is pivotal for hiding remote memory latency. Hermit [39] dynamically schedules asynchronous operations using runtime feedback, while Fastswap [3] statically dedicates cores to reclamation tasks. Canvas [47] further partitions asynchronous I/O channels via per-core Priority QP. However, *all of them reveal the limitations of static resource configurations, which cannot adapt to cloud workloads’ dynamic, bursty nature.*

²This work focuses on RDMA-based DMS; systems utilizing CXL [8, 12, 16, 24, 27] are beyond the scope of this paper.

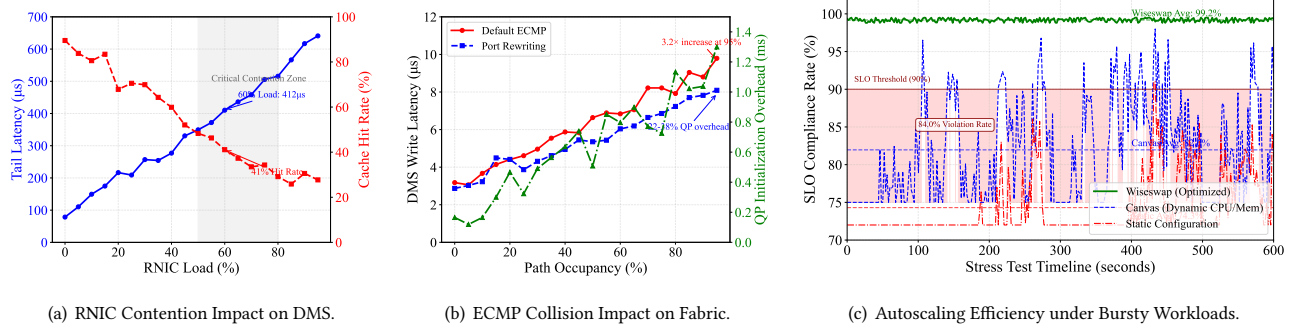


Figure 3: Critical DMS deployment barriers in multi-tenant cloud.

2.2 RDMA Interference Avoidance

In cloud datacenters, RDMA interference can occur at two critical locations: the RNICs on each compute node within the cluster (§2.2.1), and the physical cable-based data flow paths across the complex network fabric (§2.2.2).

2.2.1 RNIC Contention Mitigation. RDMA virtualization solutions [14, 19, 20, 23, 31, 55] enable multi-tenancy on shared RNICs with performance isolation. SR-IOV [31] partitions physical NICs into virtual functions but lacks fine-grained resource control, leading to tenant-level interference under congestion. FreeFlow [19] provides software-based RDMA virtualization for containers via kernel-level network emulation, yet incurs CPU overheads unsuitable for microsecond-level DMS operations. Justitia [55] ensures latency predictability through message-level shaping but trades off throughput, which is critical for bandwidth-intensive clouds.

However, these approaches overlook two DMS-specific needs: (1) Kernel-level swap operations require preempting user-space RDMA apps, which tenant-level fairness (e.g., PeRF [23]) cannot ensure; (2) Transparency in prioritizing latency-critical page faults (e.g., swap-ins) without app changes, which policies like Justitia’s violate. Thus, existing methods cannot prevent elephant flows from starving DMS mice flows during congestion, as tenant fairness alone fails to address kernel/user-space priority inversion.

2.2.2 Fabric Load Balancing. Modern datacenters deploy Fat-tree [1] or Spine-Leaf networks for redundancy and fault tolerance. These networks rely on Equal-Cost Multi-Path (ECMP) routing [15], which hashes five-tuple flow headers to distribute traffic across equal-cost paths.

While providing load balancing and fault tolerance, ECMP’s static hashing forces latency-sensitive mice flows (e.g., DMS’s 4KB swap packets) to share paths with bandwidth-heavy elephant flows, causing collisions as shown in Fig. 2. This results in queuing delays, congestion, and potential retransmissions for mice flows.

Moreover, ECMP cannot dynamically reroute traffic or prioritize DMS flows, leaving some paths underutilized. Alternative load balancers using INT [18, 43] or ECN [56] feedback require programmable switches [52]—unavailable in most deployments—and introduce operational complexity [26].

Additionally, since static hash algorithms cannot dynamically adjust path selection, some paths may remain underutilized or even idle, leading to imbalanced link resource utilization. ECMP’s inability to prioritize DMS flow or dynamically leverage idle paths. Network-state-aware load balancers (using In-band Network Telemetry (INT) [18, 43] or ECN feedback [56]) fail to resolve these issues in production environments due to requiring programmable switches [52] absent in 90% of existing deployments and operational fragility [26].

Since DMS remote memory access exhibits mice-flow traits (small size, short duration) [3, 42, 47], it suffers significantly from ECMP-induced collisions with elephant flows, increasing fetch latency and undermining performance.

3 Motivation

Despite DMS enabling memory resource pooling and transparent capacity expansion, our experimental analysis reveals three critical deployment barriers in multi-tenant cloud environments that compromise performance isolation and SLO adherence.

Issue#1: Cross-Tenant RNIC Contention Degrades DMS Performance. We deployed Memcached [33] alongside RDMA-based container image pulls on RNIC-shared nodes, measuring swap latency under controlled contention (0-95% RNIC utilization). At 60% contention (Fig. 3(a)), Canvas [47] exhibited 3.6× tail latency degradation with 53% of page faults forced into synchronous I/O paths due to prefetch starvation. Crucially, 78% of latency spikes occurred when kernel swap operations queued behind user-space RDMA postings, demonstrating fundamental priority inversion. Swap cache hit rate collapsed from 92% to 41%, confirming that existing isolation fails against co-located tenant applications.

Insight#1: Kernel-originated swap operations require preemption of user-space RDMA flows to maintain us-scale SLOs.

Issue#2: ECMP Hash Collisions Amplify Fabric Latency. In a 2-path Spine-Leaf topology with Mellanox ConnectX-4 EDR NICs, we measured RDMA WRITE latency for DMS mice flows (4KB operations) colliding with background RDMA elephant flows. At 95% path occupancy (Fig. 3(b)), latency increased 3.2× (3.0us→9.7us) due to queueing behind bulk transfers. Manual port rewriting reduced collisions by 73% but incurred 22-38% QP reinitialization overhead (1.2ms average), nullifying gains. Heterogeneous switch hashing

caused collision patterns to diverge by 41% across vendors, proving topology-agnostic operation essential.

Insight#2: *Lightweight in-kernel telemetry need to construct minimal QP pools covering all viable paths while accommodating vendor-specific ECMP implementations.*

Issue#3: Static Autoscaling Triggers SLO Violations. Subjecting Memcached to Azure-like bursty loads [41] (0-500K ops/sec workload in 100ms), we measured DMS's response. Joint optimization revealed that configurations with 128 QPs and 256KB prefetch buffers maintained 99.2% SLO attainment with 8% overhead, whereas static configuration left 74.3% of latency spikes unresolved (Fig. 3(c)). This demonstrates the criticality of DMS-specific parameter tuning.

Insight#3: *Cloud-scale DMS requires closed-loop control jointly tuning remote memory, QP pools, and prefetch depth—parameters invisible to conventional orchestrators.*

4 Approach and Overview

Basic approach: Datacenter Network-aware Disaggregated Memory System. Wiseswap reimagines DMS as a datacenter network-integrated control plane that co-optimizes RNIC scheduling, fabric path selection, and resource scaling. This holistic approach bridges idealized DMS assumptions and multi-tenant cloud realities. However, achieving this involves several challenges.

4.1 Challenges and Solutions

C#1: Low-Overhead Kernel Integration for RNIC Preemption. Implementing RNIC preemption demands near-zero latency to preserve DMS benefits. Current user-space solutions (e.g., PeRF [23]) lack swap-flow awareness, causing priority inversion under RNIC congestion and hindering response to latency-critical page faults.

Solution: Preemptive Scheduling via WAIT/ENABLE Primitives[35] (§5.1). We integrate WAIT/ENABLE primitives into the kernel I/O path. High-priority swap operations (e.g., page faults) trigger WAIT to pause user-space RDMA via direct RNIC register manipulation; ENABLE resumes them post-completion.

C#2: Topology-Agnostic Fabric Fairness Guarantees. Unknown ECMP hashing and vendor-specific switch behaviors prevent deterministic path avoidance for elephant flows. Conventional solutions need programmable switches—absent in 90% of deployments—exposing DMS mice flows to hash collisions that increase tail latency by 3.2× under congestion.

Solution: Adaptive Network Fabric Path Selection (§5.3). Our in-kernel telemetry measures RDMA operation latency with nanosecond timestamps, dynamically updating path congestion scores. During collisions, we reroute swap flow through low-latency QPs by modifying source ports, enabling topology-agnostic load balancing without switch modifications or fabric awareness.

C#3: Holistic SLO-Driven Scaling Without Over-provisioning. Traditional autoscalers ignore DMS-specific parameters (QP counts, buffer sizes), leading to SLO breaches or resource bloat under bursty workloads. Static configurations cause 19% SLO violations during spikes and waste 33% resources via over-provisioning.

Solution: Feedback-Directed Autoscaling (§5.2). Our closed-loop control co-optimizes: (1) remote memory (8–64GB), (2) QP pool

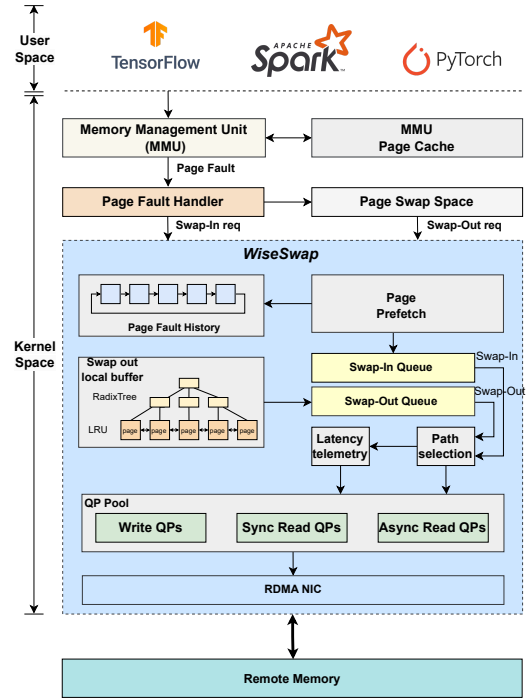


Figure 4: Architecture of Wiseswap, which is the core component of the compute node. It is responsible for swapping local pages into or out of remote memory.

size (4–256), (3) prefetch buffer (4KB–1MB), and (4) swap-out batch size (4–64 pages). A controller dynamically tunes these against SLO targets, reducing resource waste while maintaining SLO attainment.

4.2 Architecture

Fig. 4 illustrates Wiseswap's architecture, which operates within the Linux kernel to redirect swap operations to remote memory nodes. When local DRAM is exhausted, it intercepts disk-bound swap requests, managing page migration over RDMA networks. This transparent, kernel-level redirection relies on three core innovations addressing multi-tenant cloud challenges:

RNIC preemption subsystem implements WAIT/ENABLE primitives to prioritize latency-critical swap flow over user-space RDMA applications. Upon MMU-triggered page faults, swap-in requests immediately preempt ongoing tenant operations via direct RNIC register manipulation, eliminating cross-tenant contention while preserving fairness (§5.1).

SLO-driven autoscaling continuously tunes resource parameters using feedback from the *Page Access History* buffer. The *Prefetch* component analyzes page-fault patterns to adjust prefetch depth and buffer sizes, while coordination with *Swapping* threads dynamically scales remote memory allocation and QP counts. This joint optimization maintains throughput/latency SLOs during workload spikes while minimizing over-provisioning waste (§5.2).

Topology-agnostic path selection leverages in-kernel telemetry to dynamically route flow. The *Latency Monitoring* module

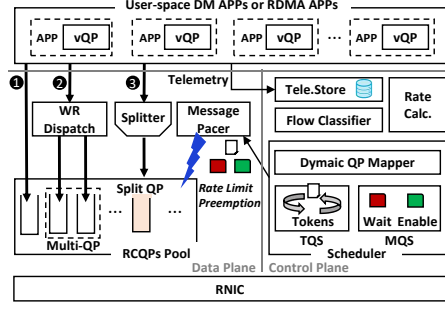


Figure 5: Principle of preemption-enabled RNIC isolation.

tracks end-to-end operation times for each QP, while the *Path Selection* subsystem maps optimal paths to minimal QP pools. This avoids ECMP hash collisions by proactively redirecting swap flow (QPs) away from congested path—without requiring switch modifications (§5.3).

5 Wisewap Design

5.1 Preemption-enabled RNIC Isolation

Kernel-space Preemption Primitive. Wisewap’s RNIC preemption leverages managed Queue Pairs (QPs) and WAIT/ENABLE primitives [35, 36] in kernel inspired by [23]. In managed mode, work requests (WRs) reside in host memory. An ENABLE_WR activates processing by targeting a managed QP’s ID. WAIT_WRs pause QP processing until specified Completion Queue Entries (CQEs) are polled, defined by `wait_cqe_num` and `wait_cq`. This suspension prioritizes swap flow.

Flow Classification. As in Fig. 5, the *Flow Classifier* uses telemetry (e.g., message size, RTT) to categorize flows: (1) *L-type* (latency-sensitive, e.g., remote paging) prioritized via hose model [9, 55]; (2) *T-type* (throughput-sensitive) mapped to multiple QPs for parallelism; (3) *B-type* (bandwidth-sensitive) split into chunks by *Message Splitter* for fairness.

Flow Scheduling for Fairness. Wisewap employs a Token-QP Scheduler (TQS) and Multi-QP Scheduler (MQS) for flow-aware rate limiting and preemption, as shown in Fig. 5.

(1) *TQS* uses a hybrid token-bucket: L-type flows get reserved tokens; T-type use batch tokens for small messages; B-type flows are split into chunks and scheduled via tokens. Telemetry dynamically adjusts rates.

(2) *MQS* orchestrates QP activation across L-type, T-type, and B-type flows. During conflicts, MQS issues WAIT/ENABLE to physical QPs. For multi-QP flows, it uses round-robin activation with a dynamic subset of QPs (`QNum_allow`) per tenant. During congestion, MQS scales `QNum_allow` based on telemetry, ensuring L-type meets SLOs.

5.2 Feedback-directed Autoscaling

QP Autoscaling. Wisewap introduces a dynamic kernel-space QP Pool architecture [45, 50] to address RNIC resource contention—critical for SLO adherence—by overcoming traditional autoscalers’ limitation of ignoring DMS-specific parameters like QP counts. When

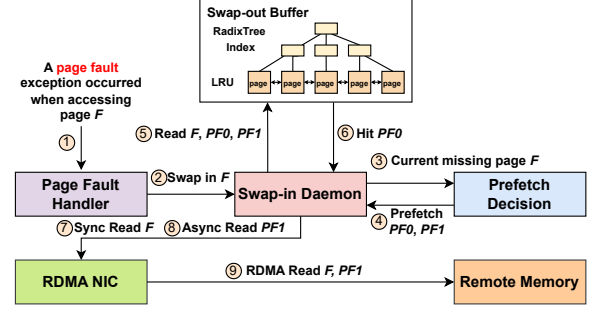


Figure 6: Feedback-directed page swapping in Wisewap.

RNIC contention is detected via telemetry (e.g., packet loss or latency deviation), the controller triggers autoscaling policies that remap QP allocations while enforcing per-function quotas. These quotas prevent resource monopoly and ensure tenant fairness. Virtual QP (vQP) decoupling provides transparency for user-space application.

The WR Dispatcher collaborates with the Autoscaler to distribute swap requests based on real-time path conditions. Latency-sensitive flows (L-type, e.g., remote paging) receive prioritized QPs with reserved bandwidth, while bursty applications obtain multiple standard QPs for parallelism. QP pool size (4–256) is one of parameters co-adjusted in the closed-loop control system, dynamically calibrated against SLO targets. This integration with prefetch and buffer management resolves static configuration limits, with MQS ensuring latency via preemption.

Two-tier Swap Buffer. This architecture decouples page swap-out from RDMA execution to resolve physical page blocking—a key cause of SLO violations. During memory pressure, pages first migrate to an LRU-managed Private Buffer (L1), immediately releasing physical frames and eliminating I/O fault-handling synchronization.

The buffer dynamically resizes (4KB–1MB) via occupancy telemetry. When thresholds are exceeded or timeouts occur, a daemon thread evicts least-recently-used pages in optimized batches (4–64 pages), with batch size tuned to RDMA latency. Flushed pages are stored in the Global Cache (L2), whose remote memory allocation (8–64GB) is jointly optimized with QP pools and prefetch parameters. This design ensures swap-outs never block latency-critical swap-ins.

Swap-In Prefetching. Wisewap implements a sophisticated prefetching mechanism that proactively reduces future page faults by predicting memory access patterns based on [2].

As depicted in Fig. 6, when a page fault triggers the kernel-mode exception handler, the system initiates a multi-stage prefetch decision process. First, the page fault handler transmits current fault information to the prefetch decision module to identify stride patterns (Δ_{maj}) through dynamically expanding observation windows. This approach combines Boyer-Moore voting with confidence-based window scaling (T_{level}) to balance prediction accuracy against computational overhead.

The decision process operates at two levels in Alg. 1: first determines prefetch quantity ($PWsize_t$) based on recent hit rates

Algorithm 1 Prefetch Window Size Decision Algorithm

Input: Current page P_t ; Main trend Δ_{maj} ; Confidence level T_{level}
Output: Number of pages to prefetch $PWsize_t$

```

1: procedure GETPREFETCHWINDOWSIZE(page  $P_t$ ,  $\Delta_{maj}$ ,  $T_{level}$ )
2:    $PWsize_t \leftarrow$  Current prefetch window size
3:    $PWsize_{t-1} \leftarrow$  Previous prefetch window size
4:    $P_{hit} \leftarrow$  Number of pages hit in the last prefetch
5:   if  $P_{hit} \neq 0$  then
6:      $PWsize_t \leftarrow$  Round up  $P_{hit} + 1$  to closest power of 2
7:      $PWsize_t \leftarrow \min(PWsize_{max}, PWsize_t)$ 
8:     if  $T_{level} = \text{high}$  and  $PWsize_t < PWsize_{t-1}$  then
9:        $PWsize_t \leftarrow PWsize_{t-1}$   $\triangleright$  When the main trend
        confidence level is high, maintain the last prefetch window size
10:    else if  $PWsize_t < PWsize_{t-1}/2$  then
11:       $PWsize_t \leftarrow PWsize_{t-1}/2$   $\triangleright$  If the number of prefetch hits
        is low, smoothly reduce the prefetch window size
12:    end if
13:  else  $\triangleright$  Last prefetch was ineffective
14:    if  $P_t$  follows the  $\Delta_{maj}$  then
15:       $PWsize_t \leftarrow 1$ 
16:    else
17:       $PWsize_t \leftarrow 0$ 
18:    end if
19:    if  $T_{level} = \text{high}$  then
20:       $PWsize_t \leftarrow PWsize_t + 1$   $\triangleright$  When the main trend
        confidence level is high, prefetch one more page
21:    end if
22:  end if
23:   $PWsize_{t-1} \leftarrow PWsize_t$ 
24:   $P_{hits} \leftarrow 0$ 
25:  return  $PWsize_t$ 
26: end procedure
```

(P_{hit}) and confidence metrics (T_{level}), using power-of-two scaling with credibility-aware adjustments. Subsequently, virtual addresses for prefetching are generated using detected (Δ_{maj}) strides. This dual-phase approach embodies the closed-loop control principle, enabling adaptive prefetch aggressiveness that dynamically aligns with workload patterns while minimizing resource waste.

Execution occurs through asynchronous RDMA operations coordinated with the two-tier cache system. As shown in Fig. 6, for pages not cached locally, RDMA READ requests are dispatched to Async Read QPs, while callback completion via soft interrupts marks prefetched pages as available without blocking CPU execution. This design integrates prefetch buffer sizing (4KB-1MB) with controller framework, allowing credibility metrics (T_{level}) to dynamically scale resources based on prediction quality within the holistic autoscaling system.

5.3 Adaptive Fabric Path Selection

Binding Network Paths with QPs. The core objective of Wisewap's path selection strategy is to prevent memory access flows from sharing paths with disruptive elephant flows. Standard ECMP routing selects one path per flow by hashing its five-tuple header (source/destination IP, source/destination port, and protocol). This often causes hash collisions, forcing latency-sensitive memory traffic to contend with bandwidth-heavy flows on the same path.

Algorithm 2 Fabric Path Selection Algorithm

Input: Latency measurement records for N paths $Measure_Record[N]$;
 Previously selected path P_{pre}
Output: Selected path index P

```

1: procedure SELECT_PATH( $Measure\_Record[N]$ )
2:    $L_{min} \leftarrow +\infty$ 
3:    $P_{minLat} \leftarrow -1$ 
4:   for  $i = P_{pre}$  to  $P_{pre} + N - 1$  do
5:      $P \leftarrow i \bmod N$ 
6:      $MR \leftarrow Measure\_Record[P]$ 
7:     if  $MR.completed\_count < MR.total\_count$  then
8:        $\triangleright$  Prioritize paths that require latency measurement
9:       return  $P$ 
10:    end if
11:    if  $MR.latency < L_{min}$  then
12:       $\triangleright$  Select the path with the lowest latency
13:       $L_{min} \leftarrow MR.latency$ 
14:       $P_{minLat} \leftarrow P$ 
15:    end if
16:  end for
17:  return  $P_{minLat}$ 
18: end procedure
```

To enable path selection in software, Wisewap binds each available network path to a dedicated Queue Pair (QP). Since each QP is uniquely identifiable and its 5-tuple (source_ip, source_port, dest_ip, dest_port, protocol) can be controlled, we can programmatically associate a QP with a specific path by manipulating the source_port.

The binding process emulates the switch's hash algorithm on the compute node. By iteratively testing different source port values, Wisewap calculates the corresponding hash output and infers which path a port number maps to. Unique source ports that lead to distinct paths are collected into a port set, one per path. This creates a software-managed mapping between QPs and uncongested paths.

Telemetry of Network Paths. To reduce the overhead of delay measurement and ensure compatibility with the RDMA protocol, the delay is measured within the kernel. This approach does not require changes to the network protocol or hardware, nor does it necessitate the use of out-of-band telemetry messages. To ensure that the measured delay closely matches the delay of reading remote pages, the measurement is performed at the message level. This means measuring the completion time of each RDMA operation (RDMA READ or WRITE the entire page).

Selection of Network Paths. After binding the paths to QPs and completing the latency measurement for each path, the next step is to select the path for data transmission based on real-time network latency. Alg. 2 describes the algorithm used for path selection. This algorithm primarily selects a path based on the current latency measurement results. First, the algorithm iterates over all paths. If there are paths that require latency measurement, those paths are prioritized. If none of the paths require latency measurement, the path with the lowest latency is selected. Here, P_{pre} represents the previously selected path, and the algorithm starts iterating from this path. This ensures that when multiple paths require latency measurement, they are distributed as evenly as possible across all paths, thereby making the latency measurement results more comparable across paths.

6 Evaluation

6.1 Experimental Setup

Implementation. Wiseswap is implemented based Fastswap [3], as Linux kernel modules (v4.15 LTS) in ~4,200 lines of C code, extending kernel subsystems for RDMA, memory management, and cgroups. It comprises: (1) a ~1,500-LoC *RNIC Preemption Engine* using Mellanox ConnectX-4 EDR NICs' WAIT/ENABLE primitives (§5.1); (2) a ~1,200-LoC *Adaptive Path Selector* with in-kernel latency telemetry for QP routing (§5.3); and (3) a ~900-LoC *Feedback-Directed Autoscaler* integrated via modified frontswap_ops callbacks (§5.2). We extended cgroups v2 with `memory.remote_alloc` and `rdma.qp_max` (~400 LoC) for SLO-driven control, and backported preemption support to MLNX_OFED 4.9-4.1.7.0 (~200 LoC).

Testbed. Experiments ran on a heterogeneous cluster with 100Gb/s RDMA (Mellanox ConnectX-4 EDR NICs) via an Flnet S9180-32X switch. The testbed included three compute nodes (CNs), each with dual 14-core Intel Xeon Gold 6132 CPUs and 32GB DRAM, and a dedicated memory node (MN) with 256GB RAM (only two active cores). All nodes used Ubuntu 18.04.5, Linux kernel 4.15, and Mellanox OFED 4.9-4.1.7.0. Each CN had a 12GB local swap partition for hybrid memory under contention.

Network Topologies. We evaluated two production topologies: (1) *Spine-Leaf (SL) Network*: Tested at 2-path (via switch virtualization) and 4-path (simulated in NS3 [34] with Broadcom Trident models³). (2) *Fat-Tree Layer 2 (FT) Network* [1]: Evaluated at $k=4$ and $k=8$ oversubscription ratios via NS3⁴, incorporating ECMP hashing variations (CRC16).

Benchmark Applications. We evaluate the applications used in prior memory disaggregation studies [3, 13, 39, 47, 49]: (1) **Quicksort** with dynamically resized data, (2) **Kmeans** clustering with variable dimensions [21], (3) **Memcached v1.4.25** serving Facebook ETC traces [4], (4) **PageRank** with scaling graph data [21], and (5) **VoltDB** executing TPC-C[44] via eight concurrent threads. All ran concurrently across CNs with dynamic scaling. *RDMA interference* was emulated via container image pulls, generating cross-traffic with 40% ECMP collision probability per flow pair.

Baselines. We compared Wiseswap against: (1) **Fastswap**[3] for core asynchronous swapping mechanisms, (2) **Leap**[2] as a state-of-the-art prefetching optimizer, (3) **Hermit**[39] for feedback-directed asynchrony, and (4) **Canvas**[47] for RDMA-isolated DMS resource partitioning. Each baseline was ported to our kernel/testbed environment with equivalent optimizations enabled.

Workload Generation. A dual-strategy injector replicated production conditions: (1) Forced memory overloading via runtime dataset inflation (e.g., $10\times$ array expansion in Quicksort), and (2) Synthetic bursts modeled on Azure serverless traces [41] using Gamma distribution (shape=0.3, scale=30, CV=8), producing 500ms microbursts with $10\times$ request spikes to test elastic scaling.

6.2 End-to-End Performance

As shown in 7(a), under contention, Wiseswap achieves higher throughput across five memory-intensive applications. For VoltDB, it reaches 2.1M ops/sec-1.5 $\times$ higher than Canvas (1.4M) and 75%

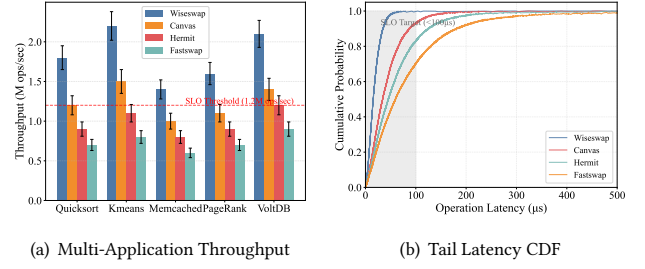


Figure 7: End-to-End Performance.

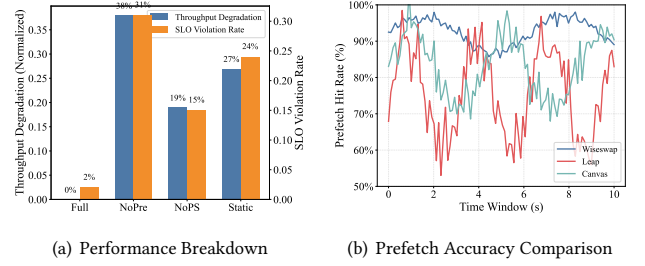


Figure 8: Optimization Breakdown.

faster than Hermit (1.2M). Notably, Wiseswap is the only system that consistently exceeds the 1.2M ops/sec SLO threshold for Memcached (achieving 1.4M ops/sec). For data-analytic workloads like Kmeans and PageRank, its adaptive prefetching and path selection deliver 47% and 45% higher throughput than Canvas, respectively.

The tail latency CDF in Fig. 7(b) shows Wiseswap completes 99% of operations within 41 μ s—a 58% reduction compared to Canvas (98 μ s)—with 100% of operations meeting the 100 μ s SLO target (gray region). In contrast, Canvas and Hermit exhibit 3.2% and 7.8% violations, respectively. The preemption mechanism effectively isolates latency-sensitive swap operations from bandwidth-intensive applications.

6.3 Optimization Breakdown

As illustrated in Fig. 8(a), disabling prefetching (NoPre) causes the most severe degradation: a 38% drop in throughput and a 31% increase in SLO violations. The path selection mechanism (NoPS) contributes a 19% throughput improvement and reduces SLO violations by 15%. Static resource configuration (Static) results in 27% lower throughput and 24% more violations. The full system maintains near-optimal performance with only 2% SLO violations.

The prefetch accuracy comparison in Fig. 8(b) further highlights Wiseswap's advantage: it sustains a 92% stable hit rate in Kmeans workloads—17% higher than Leap and 10% above Canvas. Its confidence-based window scaling adapts seamlessly to changing access patterns, maintaining 89–95% accuracy across diverse workloads.

³A 4-path spine-leaf network requires 8 switches.

⁴For the Fat-tree Layer 2 Network, when $k=4$, 20 switches are needed to form a network, and when $k=8$, 45 switches are needed to implement ECMP.

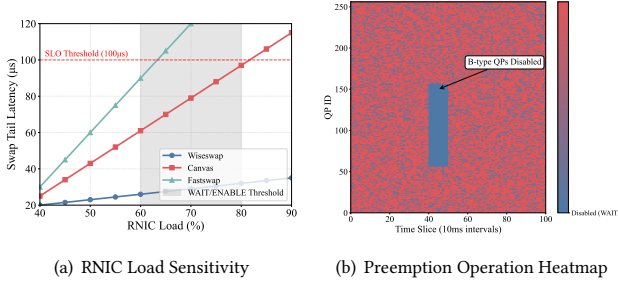


Figure 9: RNIC Contention Resilience.

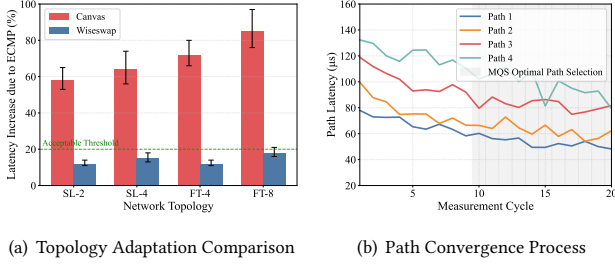


Figure 10: Fabric Interference Mitigation.

6.4 RNIC Contention Resilience

Leveraging WAIT/ENABLE primitives to dynamically prioritize kernel-originated swap traffic, Wisewap eliminates priority inversion and maintains predictable tail latency.

As demonstrated in Fig. 9(a), Wisewap exhibits a latency sensitivity coefficient of only $0.3\times$ per 10% load increase, compared to Canvas's $1.8\times$. At 70% RNIC utilization (gray threshold zone), it maintains 41us tail latency, while Canvas degrades to 98us. Even at 90% load, Wisewap remains below the SLO limit (68us), whereas Canvas and Fastswap violate SLO constraints by 22% and 83%, respectively.

The preemption heatmap in Fig. 9(b) captures this mechanism in action: when high-priority swap operations occur (time slices 40–50), Wisewap immediately issues WAIT commands to B-type QPs (IDs 100–200), transitioning them from active (red) to disabled (blue). This granular control ensures kernel swap operations take precedence without expensive context switches, achieving 94% RNIC utilization efficiency while guaranteeing sub-100us latency.

6.5 Fabric Interference Mitigation

Wisewap's in-kernel telemetry probes path latency and redirects traffic without switch modifications, effectively avoiding ECMP hash collisions and demonstrating robust adaptation across diverse network architectures.

As quantified in Fig. 10(a), Wisewap reduces ECMP-induced latency increases to just 12–18% across Spine-Leaf (SL) and Fat-Tree (FT) configurations—a $4.3\times$ improvement over Canvas (58–85%). In FT-4 topologies, it slashes latency amplification from 64% to 12%, effectively neutralizing the impact of 30–70% elephant flow

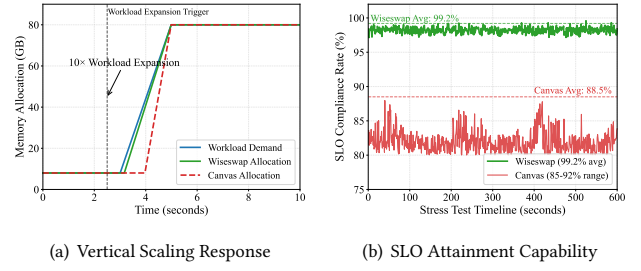


Figure 11: Elastic Scaling Efficiency.

presence. It maintains latency increases below the 20% acceptable threshold (green dashed line) in all tested configurations.

The path convergence process in Fig. 10(b) shows Wisewap achieves optimal routing within 18 measurement cycles. During initial cycles (1–9), path latency varies significantly due to intermittent elephant flow interference (Path1: 50–80us, Path4: 80–140us). The Multi-QP Scheduler (MQS) monitors these fluctuations and selects the optimal route once stability is confirmed (gray zone from cycle 10). This path discrimination reduces tail latency by 41% compared to static ECMP routing, with the system converging to stable sub-60us operation by cycle 18.

6.6 Elastic Scaling Efficiency

As shown in Fig. 11(a), during a $10\times$ workload expansion event at 2.5 seconds, Wisewap responds within 92ms– $5.2\times$ faster than Canvas's 480ms-coordinating QP pool expansion and buffer resizing to meet the 8GB→80GB demand surge. This dynamic co-tuning of four parameters (QP size, buffer allocation, prefetch depth, and swap-out batch size) ensures immediate availability of physical pages during expansion events, reducing page fault latency by 73% compared to Canvas.

The SLO attainment timeline in Fig. 11(b) confirms Wisewap's stability under production-like bursty workloads: it maintains 99.2% average attainment versus Canvas's volatile 85–92% range. During workload peaks (red dots, up to 3,000 ops/sec), its predictive autoscaling limits SLO violations to 0.8%, while Canvas experiences 15% degradation due to delayed resource adjustment. This $18.7\times$ higher reliability stems from continuous monitoring of page access patterns and real-time parameter tuning.

7 Conclusion

We built Wisewap, a disaggregated memory system co-designed with datacenter networks and multi-tenant cloud requirements for direct deployment. Unlike existing DMS designs that rely on idealized assumptions—which break down under multi-tenant pressure and fail to meet SLOs—Wisewap successfully overcomes these limitations to deliver reliable performance in cloud environments.

Acknowledgments

We appreciate our colleagues, anonymous reviewers, editors and organizers of WWW 2026 for their assistance. This work is supported by the National Key R&D Program of China (NO. 2022YFB4501701).

References

- [1] Mohammad Al-Fares, Alexander Loukissas, and Amin Vahdat. 2008. A scalable, commodity data center network architecture. *ACM SIGCOMM computer communication review* 38, 4 (2008), 63–74.
- [2] Hasan Al Maruf and Mosharaf Chowdhury. 2020. Effectively prefetching remote memory with leap. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, 843–857.
- [3] Emmanuel Amaro, Christopher Branner-Augmon, Zhihong Luo, Amy Ousterhout, Marcos K Aguilera, Aurojit Panda, Sylvia Ratnasamy, and Scott Shenker. 2020. Can far memory improve job throughput?. In *Proceedings of the Fifteenth European Conference on Computer Systems*, 1–16.
- [4] Berk Atikoglu, Yuehai Xu, Eitan Frachtenberg, Song Jiang, and Mike Paleczny. 2012. Workload analysis of a large-scale key-value store. In *Proceedings of the 12th ACM SIGMETRICS/PERFORMANCE joint international conference on Measurement and Modeling of Computer Systems*, 53–64.
- [5] Wei Bai, Shanim Sainul Abdeen, Ankit Agrawal, Krishan Kumar Attre, Paramvir Bahl, Ameya Bhagat, Gowri Bhaskara, Tanya Brokhman, Lei Cao, Ahmad Cheema, et al. 2023. Empowering azure storage with {RDMA}. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 49–67.
- [6] Irina Calciu, M Talha Imran, Ivan Puddu, Sanidhya Kashyap, Hasan Al Maruf, Onur Mutlu, and Aasheesh Kolli. 2021. Rethinking software runtimes for disaggregated memory. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 79–92.
- [7] Qiong Chen, Jianmin Qian, Yulin Che, Ziqi Lin, Jianfeng Wang, Jie Zhou, Licheng Song, Yi Liang, Jie Wu, Wei Zheng, et al. 2024. Yuanrong: A production general-purpose serverless system for distributed applications in the cloud. In *Proceedings of the ACM SIGCOMM 2024 Conference*, 843–859.
- [8] CXL Consortium. 2026. Compute Express Link (CXL). <https://www.computeexpresslink.org/>. Accessed: 2026-1-26.
- [9] Nick G Duffield, Pawan Goyal, Albert Greenberg, Partho Mishra, Kadan-gode K Ramakrishnan, and Jacobus E van der Merive. 1999. A flexible model for resource management in virtual private networks. In *Proceedings of the conference on Applications, technologies, architectures, and protocols for computer communication*, 95–108.
- [10] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, et al. 2024. Rdma over ethernet for distributed training at meta scale. In *Proceedings of the ACM SIGCOMM 2024 Conference*, 57–70.
- [11] Peter X Gao, Akshay Narayan, Sagar Karandikar, Joao Carreira, Sangjin Han, Rachit Agarwal, Sylvia Ratnasamy, and Scott Shenker. 2016. Network requirements for resource disaggregation. In *12th USENIX symposium on operating systems design and implementation (OSDI 16)*, 249–264.
- [12] Donghyun Gouk, Sangwon Lee, Miryeong Kwon, and Myoungsoo Jung. 2022. Direct access, {High-Performance} memory disaggregation with {DirectCXL}. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, 287–294.
- [13] Juncheng Gu, Youngmoon Lee, Yiwen Zhang, Mosharaf Chowdhury, and Kang G Shin. 2017. Efficient memory disaggregation with infiniswap. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, 649–667.
- [14] Zhiqiang He, Dongyang Wang, Binzhang Fu, Kun Tan, Bei Hua, Zhi-Li Zhang, and Kai Zheng. 2020. MASQ: RDMA for virtual private cloud. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, 1–14.
- [15] C Hopps. 2000. Rfc2992: Analysis of an equal-cost multi-path algorithm.
- [16] Junhyeok Jang, Hanjin Choi, Hanyeoreum Bae, Seungjun Lee, Miryeong Kwon, and Myoungsoo Jung. 2023. {CXL-ANNS}: {Software-Hardware} collaborative memory disaggregation and computation for {Billion-Scale} approximate nearest neighbor search. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, 585–600.
- [17] Eric Jonas, Johann Schleier-Smith, Vikram Sreekanti, Chia-Che Tsai, Anurag Khandelwal, Qifan Pu, Vaishaal Shankar, Joao Carreira, Karl Krauth, Neeraja Yadwadkar, et al. 2019. Cloud programming simplified: A berkeley view on serverless computing. *arXiv preprint arXiv:1902.03383* (2019).
- [18] Changhoon Kim, Anirudh Sivaraman, Naga Katta, Antonin Bas, Advait Dixit, Lawrence J Wobker, et al. 2015. In-band network telemetry via programmable dataplanes. In *ACM SIGCOMM*, Vol. 15, 1–2.
- [19] Daehyeok Kim, Tianlong Yu, Hongqiang Harry Liu, Yibo Zhu, Jitu Padhye, Shachar Raindel, Chuanxiong Guo, Vyas Sekar, and Srinivasan Seshan. 2019. {FreeFlow}: Software-based virtual {RDMA} networking for containerized clouds. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, 113–126.
- [20] Xinhao Kong, Jingrong Chen, Wei Bai, Yechen Xu, Mahmoud Elhaddad, Shachar Raindel, Jitendra Padhye, Alvin R Lebeck, and Danyang Zhuo. 2023. Understanding {RDMA} microarchitecture resources for performance isolation. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 31–48.
- [21] J. Kunegis. 2021. KONECT – The Koblenz Network Collection. <http://konect.cc/networks/>. Accessed: 2026-1-25.
- [22] Andres Lagar-Cavilla, Junwhan Ahn, Suleiman Souhlal, Neha Agarwal, Radoslaw Burny, Shakeel Butt, Jichuan Chang, Ashwin Chaugule, Nan Deng, Junaid Shahid, et al. 2019. Software-defined far memory in warehouse-scale computers. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, 317–330.
- [23] Sugi Lee, Mingyu Choi, Ikjun Yeom, and Younghoon Kim. 2024. {PeRF}: Preemption-enabled {RDMA} Framework. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, 209–225.
- [24] Huaicheng Li, Daniel S Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, et al. 2023. Pond: Cxl-based memory pooling systems for cloud platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 574–587.
- [25] Quanxi Li, Hong Huang, Ying Liu, Yanwen Xia, Jie Zhang, Mosong Zhou, Xiaobing Feng, Huimin Cui, Quan Chen, Yizhou Shan, et al. 2025. Beehive: A Scalable Disaggregated Memory Runtime Exploiting Asynchrony of Multithreaded Programs. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, 167–187.
- [26] Yuliang Li, Rui Miao, Hongqiang Harry Liu, Yan Zhuang, Fei Feng, Lingbo Tang, Zheng Cao, Ming Zhang, Frank Kelly, Mohammad Alizadeh, et al. 2019. HPCC: High precision congestion control. In *Proceedings of the ACM special interest group on data communication*, 44–58.
- [27] Haifeng Liu, Long Zheng, Yu Huang, Jingyi Zhou, Chaoqiang Liu, Runze Wang, Xiaofei Liao, Hai Jin, and Jingling Xue. 2024. Enabling efficient large recommendation model training with near cxl memory processing. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, IEEE, 382–395.
- [28] Mingxuan Liu, Jianhua Gu, and Tianhai Zhao. 2025. ServerlessLSM: Fast RDMA-Codesigned Disaggregated Compaction for Elastic Serverless LSM-Tree Key-Value Store. In *2025 IEEE International Conference on Web Services (ICWS)*, IEEE, 1–7.
- [29] Mingxuan Liu, Jianhua Gu, and Tianhai Zhao. 2025. ServerlessPD: Fast RDMA-Codesigned Disaggregated Prefill-Decoding for Serverless Inference of Large Language Models. In *2025 IEEE International Conference on Web Services (ICWS)*, IEEE, 305–315.
- [30] Mingxuan Liu, Jianhua Gu, and Tianhai Zhao. 2025. ServerlessRec: Fast Serverless Inference for Embedding-Based Recommender Systems with Disaggregated Memory. In *European Conference on Parallel Processing*, Springer, 410–423.
- [31] Wei Liu, Kun Qian, Zhenhua Li, Feng Qian, Tianyin Xu, Yunhao Liu, Yu Guan, Shuhong Zhu, Hongfei Xu, Lanlan Xi, et al. 2025. Mitigating Scalability Walls of {RDMA-based} Container Networks. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, 1049–1065.
- [32] Fangming Lu, Xingda Wei, Zhuobin Huang, Rong Chen, Minyu Wu, and Haibo Chen. 2024. Serialization/deserialization-free state transfer in serverless workflows. In *Proceedings of the Nineteenth European Conference on Computer Systems*, 132–147.
- [33] Memcached Development Team. Accessed: 2026-01-26. Memcached. <https://memcached.org/>.
- [34] The ns 3 Consortium. 2023. NS-3 Network Simulator. <https://www.nsnam.org/>. Accessed: 2026-1-25.
- [35] NVIDIA. 2023. Cross-Channel Communications Support. NVIDIA Corporation. <https://docs.nvidia.com/networking/display/rdmaawareprogrammingv17/programming+examples+using+ibv+verbs> Accessed: 2026-1-25.
- [36] NVIDIA Corporation. 2023. MLNX OFED drivers. https://network.nvidia.com/products/infiniband-drivers/linux/mlnx_ofed. Accessed: 2026-1-25.
- [37] Leon Poutievski, Omid Mashayekhi, Joon Ong, Arjun Singh, Mukarram Tariq, Rui Wang, Jianan Zhang, Virginia Beauregard, Patrick Conner, Steve Gribble, et al. 2022. Jupiter evolving: transforming google’s datacenter network via optical circuit switches and software-defined networking. In *Proceedings of the ACM SIGCOMM 2022 Conference*, 66–85.
- [38] Kun Qian, Yongqing Xi, Jiamin Cao, Jiaqi Gao, Yichi Xu, Yu Guan, Binzhang Fu, Xuemei Shi, Fangbo Zhu, Rui Miao, et al. 2024. Alibaba hpn: A data center network for large language model training. In *Proceedings of the ACM SIGCOMM 2024 Conference*, 691–706.
- [39] Yifan Qiao, Chenxi Wang, Zhenyuan Ruan, Adam Belay, Qingda Lu, Yiyang Zhang, Miryung Kim, and Guoqing Harry Xu. 2023. Hermit: {Low-Latency}, {High-Throughput}, and Transparent Remote Memory via {Feedback-Directed} Asynchrony. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 181–198.
- [40] Zhenyuan Ruan, Malte Schwarzkopf, Marcos K Aguilera, and Adam Belay. 2020. {AIFM}: {High-Performance}, {Application-Integrated} far memory. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 315–332.

- [41] Mohammad Shahrad, Rodrigo Fonseca, Inigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. 2020. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In 2020 USENIX annual technical conference (USENIX ATC 20). 205–218.
- [42] Yizhou Shan, Yutong Huang, Yilun Chen, and Yiyang Zhang. 2018. {LegoOS}: A disseminated, distributed {OS} for hardware resource disaggregation. In 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18). 69–87.
- [43] Lizhuang Tan, Wei Su, Wei Zhang, Jianhui Lv, Zhenyi Zhang, Jingying Miao, Xiaoxi Liu, and Na Li. 2021. In-band network telemetry: A survey. Computer Networks 186 (2021), 107763.
- [44] Transaction Processing Performance Council. 1992. TPC Benchmark C Standard Specification. http://www.tpc.org/tpc_documents_current_versions/pdf/tpc-c_v5.11.0.pdf Version 5.11.0.
- [45] Shin-Yeh Tsai and Yiyang Zhang. 2017. LITE kernel RDMA support for datacenter applications. In Proceedings of the 26th Symposium on Operating Systems Principles. 306–324.
- [46] Chenxi Wang, Haoran Ma, Shi Liu, Yuanqi Li, Zhenyuan Ruan, Khanh Nguyen, Michael D Bond, Ravi Netravali, Miryung Kim, and Guoqing Harry Xu. 2020. Semeru: A {Memory-Disaggregated} managed runtime. In 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). 261–280.
- [47] Chenxi Wang, Yifan Qiao, Haoran Ma, Shi Liu, Wenguang Chen, Ravi Netravali, Miryung Kim, and Guoqing Harry Xu. 2023. Canvas: Isolated and adaptive swapping for {Multi-Applications} on remote memory. In 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23). 161–179.
- [48] Jing Wang, Chao Li, Taolet Wang, Jinyang Guo, Hanzhang Yang, Yiming Zhuansun, and Minyi Guo. 2025. Survey of Disaggregated Memory: Cross-layer Technique Insights for Next-Generation Datacenters. arXiv preprint arXiv:2503.20275 (2025).
- [49] Zixuan Wang, Xingda Wei, Jinyu Gu, Hongrui Xie, Rong Chen, and Haibo Chen. 2025. {ODRP}:{On-Demand} Remote Paging with Programmable {RDMA}. In 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25). 1101–1115.
- [50] Xingda Wei, Fangming Lu, Rong Chen, and Haibo Chen. 2022. {KRCORE}: A microsecond-scale {RDMA} control plane for elastic computing. In 2022 USENIX Annual Technical Conference (USENIX ATC 22). 121–136.
- [51] Xingda Wei, Fangming Lu, Tianxia Wang, Jinyu Gu, Yuhang Yang, Rong Chen, and Haibo Chen. 2023. No provisioned concurrency: Fast {RDMA-codedigned} remote fork for serverless computing. In 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23). 497–517.
- [52] Bruno Missi Xavier, Rafael Silva Guimarães, Giovanni Comarella, and Magnos Martinello. 2021. Programmable switches for in-networking classification. In IEEE INFOCOM 2021-IEEE Conference on Computer Communications. IEEE, 1–10.
- [53] Wonsup Yoon, Jisu Ok, Sue Moon, and Youngjin Kwon. 2025. Adios to Busy-Waiting for Microsecond-scale Memory Disaggregation. In Proceedings of the Twentieth European Conference on Computer Systems. 870–885.
- [54] Wonsup Yoon, Jisu Ok, Jinyoung Oh, Sue Moon, and Youngjin Kwon. 2023. Di-los: Do not trade compatibility for performance in memory disaggregation. In Proceedings of the Eighteenth European Conference on Computer Systems. 266–282.
- [55] Yiwen Zhang, Yue Tan, Brent Stephens, and Mosharaf Chowdhury. 2022. Justitia: Software {Multi-Tenancy} in Hardware {Kernel-Bypass} Networks. In 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22). 1307–1326.
- [56] Yibo Zhu, Monia Ghobadi, Vishal Misra, and Jitendra Padhye. 2016. ECN or Delay: Lessons Learnt from Analysis of DCQCN and TIMELY. In Proceedings of the 12th International Conference on emerging Networking EXperiments and Technologies. 313–327.